

UT3 – Programación de comunicaciones en red – EJERCICIOS 4 – 5 – 6

Programación de comunicaciones en red

EJERCICIOS 4 – 5 – 6



UT3 – Programación de comunicaciones en red – EJERCICIOS 4 – 5 – 6

EJERCICIO 4: Chat Simple de Una Sola Sala

Objetivo General

Crear un **chat** en el que varios clientes puedan conectarse a un **servidor** y comunicarse en **una única sala**. Cuando un cliente envía un mensaje, el servidor lo **retransmite a todos los clientes** conectados.

Puntos clave

1. Servidor de Chat:

- Utiliza un `ServerSocket` para escuchar en un puerto determinado (p. ej., 5003).
- Al recibir una conexión (vía `accept()`), crea un hilo (o `Thread`) para atender a ese cliente. De esta forma, el servidor podrá gestionar múltiples clientes concurrentemente.
- Se necesita una **lista compartida** (o estructura de datos similar) para **almacenar** los flujos de salida (`PrintWriter`) de cada cliente.
- Cuando un cliente envía un mensaje, el servidor lo leerá y **lo transmitirá a todos los clientes** utilizando esa lista. Este proceso se conoce como **difusión (broadcast)**.
- Es esencial **sincronizar** los accesos a la lista compartida (por ejemplo, utilizando `synchronized` o estructuras seguras para hilos como `CopyOnWriteArrayList`) para evitar problemas de concurrencia.

2. Cliente de Chat:

- Se conecta mediante un `Socket` al mismo puerto del servidor (p. ej., `new Socket("localhost", 5003)`).
- Tiene dos responsabilidades principales:
 1. **Enviar mensajes** al servidor: leer desde la consola local y enviar (usando `OutputStream` o un `PrintWriter`).
 2. **Recibir mensajes** del servidor: el servidor reenvía los mensajes de todos los usuarios, por lo que el cliente debe escuchar y mostrar en pantalla esos mensajes.
- Para lograr la recepción de mensajes en tiempo real, conviene usar un **hilo adicional** que lea de manera continua del `InputStream` asociado al socket y muestre por consola lo que reciba.

3. Consideraciones:

- El servidor puede anunciar que un nuevo cliente se ha conectado o que un cliente se ha desconectado, para mejorar la experiencia de chat.
- Si un cliente envía “/salir” (o cualquier comando para salir), se podría cerrar el socket de ese cliente y remover su `PrintWriter` de la lista.
- Manejo de excepciones (`IOException`) al momento de leer/escribir.
- Cerrar los recursos (socket, flujos) de forma ordenada.

4. Pistas Teóricas:

- `ServerSocket serverSocket = new ServerSocket(5003);`
- `Socket clientSocket = serverSocket.accept(); // Bloquea hasta que un cliente se conecte`

UT3 – Programación de comunicaciones en red – EJERCICIOS 4 – 5 – 6

- Para gestionar hilos: `Thread t = new Thread(new Runnable() { ... });`
- Uso de una colección global (por ejemplo `static List<PrintWriter> listaClientes = ...;`) que almacena los flujos de salida a cada cliente.
- Acciones de difusión:

```
synchronized(listaClientes) {  
    for (PrintWriter pw : listaClientes) {  
        pw.println(mensaje);  
    }  
}
```

Esqueleto de Código

Servidor (ChatSimpleServidor.java)

```
public class ChatSimpleServidor {  
    // Estructura para almacenar PrintWriters de clientes, por ejemplo:  
    // private static List<PrintWriter> listaClientes = ...  
  
    public static void main(String[] args) {  
        // 1. Crear ServerSocket en un puerto, p.ej. 5003  
        // 2. Bucle infinito aceptando conexiones (accept())  
        // 3. Por cada conexión, crear un Thread para manejar la comunicación con ese  
cliente  
        // - En el Runnable, leer en un bucle los mensajes del cliente  
        // - Hacer broadcast a los demás  
        // 4. Manejar cierre de conexiones y remover el PrintWriter del cliente de la  
lista  
    }  
  
    // Método (opcional) para enviar un mensaje a todos los clientes (broadcast)  
    // private static void broadcast(String mensaje) { ... }  
}
```

Cliente (ChatSimpleCliente.java)

```
public class ChatSimpleCliente {  
    public static void main(String[] args) {  
        // 1. Conectarse al servidor con un Socket (localhost, 5003)  
        // 2. Preparar un hilo que reciba mensajes del servidor y los imprima en  
consola  
        // 3. En un bucle, leer mensajes de la consola local y enviarlos al servidor  
        // 4. Manejar cierre de la conexión al escribir un comando de salida  
    }  
}
```

EJERCICIO 5: Servidor de Noticias “Breaking News”

Objetivo General

Implementar un **servidor** que ofrezca **categorías** de noticias (por ejemplo, “Deportes”, “Tecnología”, “Entretenimiento”) y que el cliente, tras conectarse, **seleccione** una categoría y reciba los titulares correspondientes.

Puntos clave

1. Servidor de Noticias:

- De nuevo, se utiliza un `ServerSocket` para escuchar en un puerto (p. ej., 5004).
- Una vez que un cliente se conecta, el servidor le **envía un menú de opciones** (p. ej., “1) Deportes, 2) Tecnología, 3) Entretenimiento, 4) Salir”).
- El cliente elige una opción (envía “1”, “2”, “3”, etc.), el servidor interpreta esa opción y responde con las **noticias o titulares** correspondientes.
- Se puede emplear una estructura de datos interna (por ejemplo, un `Map` o `switch-case`) que asocia cada categoría con una lista de titulares.

2. Cliente de Noticias:

- Se conecta con un `Socket`.
- Lee el menú de categorías que envía el servidor (varias líneas).
- El usuario elige la categoría, el cliente envía esa elección al servidor.
- Muestra las “noticias” recibidas y finaliza (o vuelve a preguntar al usuario si desea otra categoría, según se diseñe el flujo).

3. Consideraciones:

- Un enfoque básico sería que **cada conexión** atienda **una sola** consulta de noticias, luego se cierra. O se puede mantener la conexión abierta para que el usuario consulte varias veces.
- Prever que el usuario pueda introducir opciones no válidas (¿qué hace el servidor en ese caso?).
- Cerrar la conexión cuando la opción sea “salir”.

4. Pistas Teóricas:

- Podrás usar métodos como `out.println(...)` para enviar **varias líneas** con el menú. El cliente usará `readLine()` repetidamente para leerlas.
- Gestión de la entrada de la opción elegida, por ejemplo:

```
String opcion = in.readLine();
switch (opcion) {
    case "1":
        // noticias de deportes
        break;
    // ...
}
```

UT3 – Programación de comunicaciones en red – EJERCICIOS 4 – 5 – 6

- Al finalizar, `socket.close()` para cerrar la conexión.

Esqueleto de Código

Servidor (ServidorNoticias.java)

```
public class ServidorNoticias {
    public static void main(String[] args) {
        // 1. Crear ServerSocket (puerto 5004)
        // 2. Bucle infinito que acepte conexiones (accept())
        // 3. Por cada cliente:
        //     - Enviar menú de categorías
        //     - Leer opción del cliente
        //     - Enviar noticias de la categoría correspondiente
        //     - Cerrar o mantener la conexión según la lógica deseada
    }

    // Método (opcional) para obtener las noticias de una categoría:
    // private static String getNoticias(String categoria) { ... }
}
```

Cliente (ClienteNoticias.java)

```
public class ClienteNoticias {
    public static void main(String[] args) {
        // 1. Conectarse al servidor con un Socket (localhost, 5004)
        // 2. Leer el menú que envía el servidor (varias líneas)
        // 3. Introducir la opción por consola y enviarla al servidor
        // 4. Recibir y mostrar los titulares
        // 5. Cerrar la conexión o preguntar si se quiere más noticias
    }
}
```

EJERCICIO 6: Transmisión de Archivos Pequeños “Photo Sender”

Objetivo General

Desarrollar una **aplicación** para **transferir archivos** (por ejemplo, fotografías) desde un **servidor** a un **cliente**. El cliente solicita un **nombre de archivo**, y el servidor lo envía en **modo binario**.

Puntos clave

1. Servidor de Archivos:

- Continúa con el patrón de `ServerSocket` para escuchar.
- El **cliente** envía el **nombre del archivo** que desea (p. ej., “foto1.jpg”).
- El servidor busca ese archivo en su sistema de ficheros (por ejemplo, en la misma carpeta donde se ejecuta o en otra ruta especificada).
- Si el archivo existe, lo **lee en binario** (usando `FileInputStream`, `BufferedInputStream`) y lo transmite al cliente a través del `OutputStream`.
- Si no existe, podría enviar un mensaje de error o simplemente no enviar nada.

2. Cliente de Archivos:

- Se conecta con un `Socket`.
- Pide al usuario el nombre del archivo que desea.
- Envía ese nombre al servidor (como cadena).
- **Recibe** los bytes del archivo y los escribe en un `FileOutputStream` local, para guardarlo con un nombre (por ejemplo, “descargado_foto1.jpg”).
- Mostrar mensajes para indicar que la descarga finalizó con éxito.

3. Consideraciones:

- **Modo binario:** a diferencia de los ejercicios anteriores (principalmente de texto), aquí se transmiten bytes **sin interpretación** de caracteres.
- Aun así, el paso de un **nombre de archivo** se puede hacer en modo texto (con `PrintWriter` o `BufferedWriter`). Lo importante es que el **contenido del archivo** se envíe en binario.
- El **tamaño** del archivo puede desconocerse de antemano. El cliente debe leer en un **bucle** hasta que se retorne -1 (indicando que no hay más bytes que recibir).
- Cerrar flujos para evitar archivos corruptos.

4. Pistas Teóricas:

- Lectura del archivo en el servidor:

```
FileInputStream fis = new FileInputStream(nombreArchivo);  
BufferedInputStream bis = new BufferedInputStream(fis);
```

UT3 – Programación de comunicaciones en red – EJERCICIOS 4 – 5 – 6

- Envío al cliente:

```
OutputStream out = socket.getOutputStream();
byte[] buffer = new byte[4096];
int bytesLeidos;
while ((bytesLeidos = bis.read(buffer)) != -1) {
    out.write(buffer, 0, bytesLeidos);
}
```

- Recepción en el cliente y guardado:

```
InputStream in = socket.getInputStream();
FileOutputStream fos = new FileOutputStream("descargado_" +
nombreArchivo);
...
```

- Comprobar la existencia del archivo en el servidor (new
File(nombreArchivo).exists()).

Esqueleto de Código

Servidor (ServidorPhotoSender.java)

```
public class ServidorPhotoSender {
    public static void main(String[] args) {
        // 1. Crear ServerSocket (puerto 5005)
        // 2. Aceptar conexiones (accept())
        // 3. Leer (en modo texto) el nombre del archivo solicitado
        // 4. Buscar el archivo en el sistema local
        // 5. Si existe, abrir FileInputStream y enviar su contenido binario
        // 6. Cerrar flujos y socket
    }
}
```

Cliente (ClientePhotoSender.java)

```
public class ClientePhotoSender {
    public static void main(String[] args) {
        // 1. Conectarse al servidor con Socket (localhost, 5005)
        // 2. Pedir al usuario el nombre del archivo y enviarlo
        // 3. Recibir el contenido binario del archivo
        // 4. Guardarlo en un FileOutputStream local
        // 5. Cerrar flujos y socket
    }
}
```

UT3 – Programación de comunicaciones en red – EJERCICIOS 4 – 5 – 6

Resumen y sugerencias

- **Ejercicio 4 (Chat Simple):** Explora el manejo de **múltiples clientes concurrentes** (hilos), el patrón **broadcast** y la necesaria coordinación de recursos compartidos (lista de conexiones).
- **Ejercicio 5 (Servidor de Noticias):** Practica la **interacción secuencial** de pregunta-respuesta con menús, la **organización de datos** (categorías y titulares) y la **lógica condicional** en el servidor.
- **Ejercicio 6 (Transmisión de Archivos):** Profundiza en el **trabajo con flujos binarios**, la diferencia entre E/S de texto y E/S binaria, y la **gestión de archivos** (encontrar y enviar/recibir un archivo).

Al combinarlos, obtienen una visión amplia de cómo la **API de Java** para **sockets** (ServerSocket, Socket) y los **flujos de E/S** (InputStream, OutputStream, FileInputStream, FileOutputStream, etc.) permiten construir aplicaciones de red robustas y adaptadas a diversas necesidades.

Sugerencias

- **Personalizar** cada ejercicio (por ejemplo, añadir manejo de errores, gestión de tiempo de espera, interfaz gráfica sencilla, etc.).
- Discutir los **riesgos de seguridad** (p. ej., permitir que un cliente solicite archivos arbitrarios del servidor).
- Presentar ejemplos reales donde se requieren estos patrones básicos de comunicación (chats corporativos, sistemas de actualización de noticias, FTP simplificado, etc.).